

Secure Coding Review Report

Name: Ankita Bilgaiya

CodeAlpha Cyber Security Internship

1. Abstract

This report presents a secure coding review of a sample login application. The review was conducted to identify common security weaknesses, evaluate their impact, and recommend mitigation measures. Secure coding practices help reduce vulnerabilities that attackers can exploit. The assessment focuses on authentication, input validation, password management, and database interaction. By applying industry-standard security controls, developers can improve application reliability, confidentiality, and integrity. This report highlights major findings and provides practical recommendations for strengthening application security.

2. Introduction

Secure coding is the practice of developing software in a way that minimizes vulnerabilities and protects systems from cyber threats. Modern web applications frequently face attacks such as SQL injection, cross-site scripting, credential theft, and unauthorized access. Developers must follow secure coding guidelines throughout the software development lifecycle. A secure coding review helps identify weaknesses before deployment and reduces the likelihood of security incidents. The reviewed application is a simple login system used to demonstrate common security issues and their solutions.

3. Objectives

- Identify security vulnerabilities in the application.
- Analyze risks associated with insecure coding practices.
- Evaluate the impact of discovered vulnerabilities.
- Recommend mitigation techniques and best practices.
- Promote awareness of secure software development.
- Improve application resilience against cyber attacks.

4. Security Findings

Hardcoded Passwords: Credentials stored directly in source code can be exposed through repositories or application files.

SQL Injection: Unsanitized user input may allow attackers to manipulate database queries and gain unauthorized access.

Weak Password Policies: Short or predictable passwords increase the risk of brute-force attacks.

Insufficient Input Validation: Improper validation may lead to injection attacks and unexpected behavior.

Lack of Error Handling: Detailed error messages can reveal sensitive system information to attackers.

5. Recommendations

Use strong password hashing algorithms such as bcrypt or Argon2. Implement prepared statements and parameterized queries to prevent SQL injection. Validate and sanitize all user inputs before processing. Enforce strong password policies, including minimum length and complexity requirements. Apply role-based access control and multi-factor authentication where appropriate. Conduct regular security testing, code reviews, and vulnerability assessments. Keep software dependencies updated and follow secure development standards.

6. Conclusion

Secure coding is essential for protecting applications, user data, and organizational resources. The identified vulnerabilities demonstrate how small coding mistakes can create significant security risks. Implementing the recommended controls can greatly improve application security and reduce the likelihood of successful attacks. Continuous monitoring, testing, and developer awareness are necessary to maintain a strong security posture. Organizations should integrate secure coding practices into every phase of software development to build reliable and trustworthy applications.